Testing practice and course synthesis

CSC103 Programming Fundamentals / Lecture 32

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Design tests for a marks-file summary: normal data, one mark, threshold 50, empty file, malformed token and out-of-range mark.

Use a pure mean method that rejects empty or out-of-range data.

Learning objectives

- Organise parameterised and regression tests
- Distinguish unit and integration checks
- Test a program that reads and summarises marks

CLO-4
Parameterised tests
Isolation and fixtures
Testing an integrated program
Coverage and final review

Parameterised tests

- A parameterised test applies one behaviour check to several cases.
- @CsvSource supplies small literal datasets.
- Each case should add a reason to test.

Example

```
@ParameterizedTest
@CsvSource({"2,5,5", "-4,-2,-2", "3,3,3"})
void maximumCases(int a, int b, int expected) {
  assertEquals(expected, Numbers.max(a,b));
}
```

Isolation and fixtures

- Tests should avoid shared mutable state.
- Temporary directories isolate file tests.
- Resource cleanup matters even when an assertion fails.

Example

```
@TempDir Path temp;
Path file = temp.resolve("marks.txt");
// Write a controlled fixture for this test.
```

Testing an integrated program

- Separate parsing, validation and calculation.
- Test pure calculations with small unit tests.
- Use file fixtures to check the whole reading-and-summary path.

Example

```
Fixture: "40 50 60 70"
Expected mean: 55.0
Expected passes: 3
Invalid fixture: "50 bad 60"
```

Coverage and final review

- Line coverage says which lines executed, not whether outcomes were checked well.
- A small changed condition can reveal weak tests.
- Review algorithms, control flow, arrays and error handling together.

Example

```
Mutation thought experiment:
Change mark >= 50 to mark > 50.
Which test must fail?
Answer: the case at exactly 50.
```

A test matrix connects requirements to evidence

- Each requirement should have an observable result and a corresponding test.
- Boundary cases expose comparisons that ordinary values may never challenge.
- Failure cases should check the intended exception or message.

Example

```
Requirement: 50 counts as a pass.
Test input: {50}
Expected passing count: 1
```

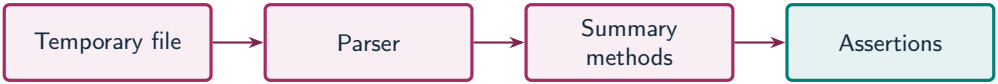
A file fixture isolates an integration test

- A fixture supplies controlled input data for a test.
- A temporary directory prevents collisions with a user's real files.
- The test then checks the result of file reading and calculation together.

Example

```
Write "40 50 60 70" to a temporary file.
Read it through the production method.
Assert mean 55.0 and passing count 3.
```

Integration tests cross component boundaries



What to notice

An integration test checks that collaborating components agree on data and behaviour.

Key distinctions

Requirement	Test input	Expected evidence
Mean calculation	40 50 60 70	55.0
One-record handling	80	Mean 80.0
Passing threshold	50	Pass count 1
Empty mean policy	Empty file	Exception for mean
Format validation	50 bad 60	Parsing-related exception
Range validation	101	Range exception

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
int[] marks = {40, 50, 60, 70};
int sum = 0, passes = 0;
for (int mark : marks) {
  sum += mark;
  if (mark >= 50) passes++;
}
System.out.println((double) sum / marks.length);
System.out.println(passes);
```

First example: result

55.0
3
A boundary test at 50 distinguishes >= from >.

Testing an integrated program
Coverage and final review

Guided practice

Design tests for a marks-file summary: normal data, one mark, threshold 50, empty file, malformed token and out-of-range mark.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Use a pure mean method that rejects empty or out-of-range data.
- Use a pass-count method with the inclusive threshold 50.
- Compare a normal dataset with a single mark exactly on the boundary.

The next program uses fixed inputs so its result can be reproduced.

Complete solution (1/2)

```
public class Solution32 {
  public static void main(String[] args) throws Exception {
    int[] marks = {40, 50, 60, 70};
    System.out.println(mean(marks));
    System.out.println(passes(marks));
    System.out.println(passes(new int[]{50}));
  }
  static double mean(int[] marks) {
    if (marks.length == 0) throw new IllegalArgumentException("empty");
    long sum = 0;
    for (int mark : marks) {
```

Complete solution (2/2)

```
      if (mark < 0 || mark > 100) throw new IllegalArgumentException("range");
      sum += mark;
    }
    return (double) sum / marks.length;
  }
  static int passes(int[] marks) {
    int count = 0;
    for (int mark : marks) if (mark >= 50) count++;
    return count;
  }
}
```

Execution trace

Test	Expected	If >= becomes >	Detects change?
Passes in {40,50,60,70}	3	2	Yes
Passes in {80}	1	1	No
Passes in {50}	1	0	Yes

Follow the changing state in execution order.

Solution result

55.0
3
1

Why this result follows
Compare a normal dataset with a single mark exactly on the boundary.

A common incorrect approach

A suite executes every line, but never checks the calculated average. Is the calculation verified?

Example

```
No. Executed lines can still produce incorrect results. Add explicit assertions with
    independently calculated expected averages.
```

Boundary and failure checks

Test 49, 50 and 51 as separate one-mark datasets.

Normal: 40 50 60 70 gives mean55 and passes3. One 80 gives mean80 and passes1. One 50 passes. Empty data rejects a mean. "bad" fails parsing. -1 or 101 fails validation.

A complete file integration test (1/2)

```java
package edu.cui.csc103;

import java.nio.file.*;
import java.nio.charset.StandardCharsets;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;
import static org.junit.jupiter.api.Assertions.*;

class FileSummaryTest {
  @TempDir Path folder;
```

A complete file integration test (2/2)

```
  @Test
  void summarisesAControlledFile() throws Exception {
    Path file = folder.resolve("marks.txt");
    Files.writeString(file, "40 50 60 70", StandardCharsets.UTF_8);
    int[] marks = MarksFile.read(file);
    assertArrayEquals(new int[]{40,50,60,70}, marks);
    assertEquals(55.0, Numbers.mean(marks), 1e-9);
    assertEquals(3, Numbers.passes(marks));
  }
}
```

Check your understanding

Which tests should fail if >=50 becomes >50? What does a file integration test cover beyond a pure unit test?

Write a prediction and explain the rule that supports it.

Answer and explanation

A case containing a mark exactly 50 with a pass-count assertion. The integration test also checks file opening, encoding, token parsing and cooperation between methods.

No. Executed lines can still produce incorrect results. Add explicit assertions with independently calculated expected averages.

Compare cases and predict outcomes

Fixture	Expected outcome	Boundary exercised
40 60	Mean 50.0; passes 1	File through summary
50 bad	Parsing failure	Malformed file token
Empty file	Empty array; mean rejected	No-data contract

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Plan an integration test for a temporary file containing 40 and 60. List the assertions that distinguish successful reading from successful calculation.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
Parsed array: assertArrayEquals(new int[]{40, 60}, marks)
Mean: assertEquals(50.0, Numbers.mean(marks), 1e-9)
Passes: assertEquals(1, Numbers.passes(marks))
```

- The array assertion checks parsing and record order.
- The mean and pass assertions check the downstream computation.
- Use an isolated temporary directory so tests do not depend on a personal file.

Integration checks for student-record files

- A temporary fixture isolates the test from personal files and previous runs.
- Check parsing separately from an aggregate calculation.
- Malformed fields and out-of-range scores are distinct failure cases.

Source: supplied ISB campus slides. See speaker notes and ISB_Source_Map.md.

Worked problem: Integration checks for student-record files

Task

Write the two source records to a temporary UTF-8 file, parse the scores, and assert scores {90,85} and mean 87.5.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/2)

```
public class IsbLecture32 {
  public static void main(String[] args) throws Exception {
    java.nio.file.Path file = java.nio.file.Files.createTempFile("isb-test-", ".txt");
    java.nio.file.Files.writeString(file, "John T Smith 90\nEric K Jones 85\n",
      java.nio.charset.StandardCharsets.UTF_8);
    int[] scores = readScores(file);
    org.junit.jupiter.api.Assertions.assertArrayEquals(new int[]{90, 85}, scores);
    double mean = (scores[0] + scores[1]) / 2.0;
    org.junit.jupiter.api.Assertions.assertEquals(87.5, mean, 1e-9);
    System.out.println("Record integration passed");
  }
  static int[] readScores(java.nio.file.Path file) throws java.io.IOException {
    java.util.List<String> lines = java.nio.file.Files.readAllLines(
```

Complete ISB example (2/2)

```
      file, java.nio.charset.StandardCharsets.UTF_8);
    int[] scores = new int[lines.size()];
    for (int i = 0; i < lines.size(); i++) {
      String[] fields = lines.get(i).trim().split("\\s+");
      if (fields.length != 4) throw new IllegalArgumentException("four fields required");
      int score = Integer.parseInt(fields[3]);
      if (score < 0 || score > 100) throw new IllegalArgumentException("score range");
      scores[i] = score;
    }
    return scores;
  }
}
```

Worked trace and expected result

Fixture	Expected parsing	Expected next step
Two source records	[90, 85]	Mean 87.5
John T Smith bad	Reject non-integer	No aggregate
John T Smith 101	Reject range	No aggregate

Expected console output

```
Record integration passed
```

Extend the ISB example

Practice

Why is checking only mean 87.5 weaker than also checking the parsed array?

Explain your reasoning

Different incorrect arrays can share the same mean. The array assertion checks record count, values and order.

Lecture summary

- parameterised and regression tests
- unit and integration checks
- a program that reads and summarises marks

- Use a pure mean method that rejects empty or out-of-range data.
- Use a pass-count method with the inclusive threshold 50.
- Compare a normal dataset with a single mark exactly on the boundary.

After-class practice

Prepare a small marks-analysis project with methods, arrays, file I/O and a documented test suite. Use the companion project as a starting point.

Syllabus reading
Reference material: JUnit Jupiter
Next: course revision and final-exam preparation